

Lista Teórica 09 — Gabarito

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Aula 09 — Máquinas de Vetores de Suporte (SVM)

Exercício 1 — a margem, na mão. Quatro observações em $\mathbb{R}^2$, linearmente separáveis:

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
$\mathbf{x}_i$	(3, 1)	(3, 3)	(1, 1)	(1, 3)
y_i	+1	+1	-1	-1

Lembre que o hiperplano de margem máxima é escrito na forma canônica $\mathbf{w}^\top \mathbf{x} + b = 0$ com $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$ para todo i , e que a margem de cada lado vale $1/\|\mathbf{w}\|$.

- Desenhe os quatro pontos e determine, por inspeção, o hiperplano de margem máxima. Escreva $\mathbf{w}$ e b na forma canônica e calcule a margem.
- Quais pontos são vetores de suporte?
- Acrescente $E = (5, 2)$ com $y_E = +1$. O hiperplano muda? Justifique sem refazer a conta.
- Agora mova A de (3, 1) para (2, 5; 1), mantendo os demais. Qual é o novo hiperplano e a nova margem? Quais são agora os vetores de suporte?

Solução

(a) Os positivos estão todos em $x_1 = 3$ e os negativos em $x_1 = 1$; a coordenada x_2 não distingue nada. O hiperplano que mais se afasta dos dois grupos é a reta vertical no meio, $x_1 = 2$.

Na forma canônica precisamos de $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$ nos pontos mais próximos. Tomando $\mathbf{w} = (1, 0)$ e $b = -2$:

$$A : (1)(3 - 2) = 1, \quad B : (1)(3 - 2) = 1, \quad C : (-1)(1 - 2) = 1, \quad D : (-1)(1 - 2) = 1.$$

Todos valem exatamente 1, como exige a forma canônica. A margem é

$$\frac{1}{\|\mathbf{w}\|} = \frac{1}{1} = 1$$

de cada lado, ou seja uma faixa de largura total 2, entre $x_1 = 1$ e $x_1 = 3$.

(b) **Todos os quatro.** Vetor de suporte é o ponto que satisfaz $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$, isto é, que *toca* a margem — e aqui os quatro tocam.

Atenção: o que o scikit-learn vai dizer

Se você conferir com `SVC(kernel="linear", C=1e6)` e imprimir `support_vectors_`, vão aparecer **dois** pontos, não quatro. Não é erro: nesta configuração simétrica a solução *dual* não é única, e o otimizador devolve um conjunto mínimo de multiplicadores que já reproduz o mesmo $\mathbf{w}$. A definição geométrica (pontos sobre a margem) e a computacional (pontos com multiplicador de Lagrange não nulo) coincidem no caso genérico e podem divergir em casos degenerados como este.

(c) **Não muda.** Para $E = (5, 2)$ temos $y_E(\mathbf{w}^\top \mathbf{x}_E + b) = 5 - 2 = 3 > 1$: o ponto está *fora* da margem, do lado certo. A solução do problema de margem máxima depende apenas dos pontos que tocam a margem — os demais têm multiplicador de Lagrange zero e poderiam ser apagados do conjunto de dados sem alterar nada.

É a propriedade que dá nome ao método e o distingue, por exemplo, do LDA, cuja fronteira depende das *médias* e portanto de todas as observações. Mover E para $(50, 2)$ também não mudaria nada; mover A um pouco muda tudo — é o item seguinte.

(d) Agora os positivos são $(2, 5; 1)$ e $(3, 3)$, e os negativos continuam em $x_1 = 1$. O ponto positivo mais próximo dos negativos é $A = (2, 5; 1)$, e o negativo mais próximo dele é $C = (1, 1)$: a distância entre eles é 1,5, na horizontal. O hiperplano de margem máxima é a mediatriz desse segmento,

$$x_1 = \frac{1 + 2,5}{2} = 1,75, \quad \text{margem} = \frac{1,5}{2} = \mathbf{0,75}.$$

Na forma canônica, $\mathbf{w} = (1/0,75, 0) = (4/3, 0)$ e $b = -1,75 \cdot 4/3 = -7/3$, de modo que $1/\|\mathbf{w}\| = 0,75$.

Confira que $D = (1, 3)$ também toca: $(-1)(4/3 - 7/3) = 1$. Já $B = (3, 3)$ dá $(1)(4 - 7/3) = 5/3 > 1$ e ficou de fora. Os vetores de suporte são portanto **A**, **C** e **D**.

Compare com o item (a): mover *um* ponto de 0,5 encolheu a margem em 25% e mudou o conjunto de vetores de suporte. A SVM de margem rígida é extremamente sensível aos pontos da fronteira — e é exatamente esse defeito que o parâmetro C do próximo exercício vem corrigir.

Exercício 2 — o que C faz, e por que ele é invertido. A SVM de margem suave resolve

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{sujeito a} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

- Descreva o que acontece quando $C \rightarrow \infty$. Existe sempre solução?
- Descreva o que acontece quando $C \rightarrow 0$. O que ocorre com $\mathbf{w}$, com a margem e com o número de vetores de suporte?
- Portanto: C grande corresponde a um modelo mais *flexível* ou mais *rígido*? Compare com o papel de λ na Ridge (Aula 02) e explique por que essa comparação confunde tanta gente.
- Ao montar uma grade de busca para C , por que se usa `np.logspace(-3, 3, 7)` e não `np.linspace(0.001, 1000, 7)`?

Solução

(a) Com $C \rightarrow \infty$, qualquer $\xi_i > 0$ paga um preço infinito, então a solução força $\xi_i = 0$ para todo i : é a **margem rígida** do Exercício 1. A margem fica a mais larga possível *sem que nenhum ponto a invada*, e poucos pontos são vetores de suporte.

Não existe sempre solução: se os dados não forem linearmente separáveis, o problema com $\xi_i = 0$ é *inviável* — não há hiperplano que satisfaça todas as restrições. É por isso que a margem suave existe: sem ela, o método simplesmente não se aplica a dados reais.

(b) Com $C \rightarrow 0$ a soma dos ξ_i deixa de custar, e o objetivo vira essencialmente $\min \frac{1}{2} \|\mathbf{w}\|^2$, cuja solução é $\mathbf{w} \rightarrow \mathbf{0}$. Então:

- $\|\mathbf{w}\| \rightarrow 0$, logo a margem $1/\|\mathbf{w}\| \rightarrow \infty$: a faixa engole todo o espaço;

- **todos** os pontos passam a estar dentro da margem, e portanto todos viram vetores de suporte;
- a fronteira deixa de depender dos dados de forma útil — o classificador degenera.

(c) C **grande** é modelo **mais flexível** (menos regularizado): ele persegue os dados de treino e aceita margens estreitas para não errar ninguém.

A confusão com λ da Ridge é natural, porque os dois parâmetros aparecem em posições análogas mas com papéis *opostos*. Escrevendo os dois objetivos lado a lado:

$$\underbrace{\|Y - X\beta\|^2 + \lambda \|\beta\|^2}_{\text{Ridge}}, \quad \underbrace{\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i}_{\text{SVM}}.$$

Na Ridge, o hiperparâmetro multiplica a **penalização**, então λ grande $\Rightarrow$ mais regularização $\Rightarrow$ mais rígido. Na SVM, ele multiplica o **erro**, então C grande $\Rightarrow$ menos regularização $\Rightarrow$ mais flexível. Dividindo o objetivo da SVM por C fica claro: $\frac{1}{2C} \|\mathbf{w}\|^2 + \sum_i \xi_i$, e $1/(2C)$ é que faz o papel de λ .

Um jeito de nunca mais errar: C **é o preço de errar**. Preço alto, o modelo não erra no treino.

(d) Porque o efeito de C é **multiplicativo**, não aditivo: o que importa é a ordem de grandeza. A diferença entre $C = 0,001$ e $C = 0,01$ é enorme (um fator de 10 na regularização), e a diferença entre $C = 900$ e $C = 1000$ é irrelevante.

Uma grade linear de 0,001 a 1000 com 7 pontos daria $\{0,001; 166,7; 333,3; 500; 666,7; 833,3; 1000\}$: seis pontos espremidos na região em que nada muda, e *nenhum* entre 0,001 e 166 — justamente o intervalo onde está o ótimo na maioria dos problemas. A grade logarítmica dá $\{10^{-3}, 10^{-2}, \dots, 10^3\}$, um ponto por ordem de grandeza.

A mesma regra vale para o α da Ridge e do Lasso, para o γ do kernel RBF e para a taxa de aprendizado do *boosting*.

Exercício 3 — o truque do kernel, explicitamente. Em $\mathbb{R}^2$, considere o kernel polinomial de grau 2

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + 1)^2.$$

- Expandir K e exibir um mapa $\varphi : \mathbb{R}^2 \rightarrow \mathbb{R}^m$ tal que $K(\mathbf{x}, \mathbf{z}) = \varphi(\mathbf{x})^\top \varphi(\mathbf{z})$. Qual é m ?
- Conte as operações necessárias para calcular $K(\mathbf{x}, \mathbf{z})$ pelos dois caminhos: pela fórmula fechada e via φ .
- Em $\mathbb{R}^p$, o espaço de atributos do kernel polinomial de grau q tem $\binom{p+q}{q}$ dimensões. Calcule para $p = 100$, $q = 3$ e para $p = 1000$, $q = 3$.
- Enuncie em uma frase o que é o “truque do kernel”, usando os números do item (c).

Solução

(a) Com $\mathbf{x} = (x_1, x_2)$ e $\mathbf{z} = (z_1, z_2)$,

$$(x_1 z_1 + x_2 z_2 + 1)^2 = x_1^2 z_1^2 + x_2^2 z_2^2 + 1 + 2x_1 x_2 z_1 z_2 + 2x_1 z_1 + 2x_2 z_2.$$

Agrupando cada termo como um produto de uma parte em $\mathbf{x}$ por uma parte em $\mathbf{z}$, lê-se diretamente

$$\varphi(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1 x_2, \sqrt{2}x_1, \sqrt{2}x_2, 1),$$

com $m = 6$. (Os $\sqrt{2}$ aparecem para que, ao multiplicar $\varphi(\mathbf{x})$ por $\varphi(\mathbf{z})$, cada termo cruzado seja contado com o fator 2 da expansão.) Confere com $\binom{2+2}{2} = 6$.

(b) Pela **fórmula fechada**: 2 multiplicações e 1 soma para o produto interno $\mathbf{x}^\top \mathbf{z}$, mais 1 soma e 1 elevação ao quadrado. Cerca de **5 operações**, e nenhuma memória extra.

Via φ : construir os dois vetores de 6 componentes (com quadrados, produtos e multiplicações por $\sqrt{2}$) e depois fazer um produto interno em $\mathbb{R}^6$, ou seja 6 multiplicações e 5 somas. Cerca de **25 operações**, mais o armazenamento dos dois vetores de dimensão 6. Em $p = 2$ e $q = 2$ a diferença é pequena; o item (c) mostra o que acontece quando não é.

(c)

$$\binom{103}{3} = \frac{103 \cdot 102 \cdot 101}{6} = \mathbf{176\,851}, \quad \binom{1003}{3} = \frac{1003 \cdot 1002 \cdot 1001}{6} = \mathbf{167\,668\,501}.$$

Com $p = 1000$ e grau 3, o espaço de atributos tem **167 milhões** de dimensões. Guardar um único $\varphi(\mathbf{x})$ em precisão dupla custaria cerca de 1,3 GB; guardar os $\varphi(\mathbf{x}_i)$ de mil observações seria impossível.

(d) O truque do kernel é: **a SVM só precisa de produtos internos entre observações, nunca das coordenadas em si** — então dá para trabalhar num espaço de 167 milhões de dimensões calculando, para cada par, um produto interno em $\mathbb{R}^{1000}$ seguido de uma soma e um cubo, *sem nunca construir esse espaço*.

O que torna isso possível é a formulação **dual** do problema: nela $\mathbf{x}$ aparece exclusivamente dentro de produtos $\mathbf{x}_i^\top \mathbf{x}_j$, que podem ser substituídos por $K(\mathbf{x}_i, \mathbf{x}_j)$. O preço é que o custo passa a ser governado por n , não por p — a matriz de Gram é $n \times n$, o que explica o custo entre $O(n^2)$ e $O(n^3)$ do treino e por que a SVM não escala para milhões de observações.

Exercício 4 — a perda hinge e a esparsidade. A SVM de margem suave é equivalente a minimizar

$$\sum_{i=1}^n \underbrace{\max(0, 1 - y_i f(\mathbf{x}_i))}_{\text{perda hinge}} + \lambda \|\mathbf{w}\|^2, \quad f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b.$$

- (a) Complete a tabela abaixo com o valor das três perdas, em função da *margem funcional* $m = yf(\mathbf{x})$.

$m = yf(\mathbf{x})$	-1	0	0,5	1	2
0-1: $\mathbf{1}\{m < 0\}$					
hinge: $\max(0, 1 - m)$					
logística: $\log_2(1 + e^{-m})$					

- (b) Para quais valores de m a perda hinge vale exatamente zero? Relacione com o conceito de vetor de suporte.
- (c) A perda logística vale zero para algum m finito? Que consequência isso tem sobre *quais* observações influenciam o ajuste de uma regressão logística?
- (d) Por que não se minimiza diretamente a perda 0-1, que é a que realmente interessa?

Solução

(a)

$m = yf(\mathbf{x})$	-1	0	0,5	1	2
0-1	1	0	0	0	0
<i>hinge</i>	2	1	0,5	0	0
logística	1,8946	1,0000	0,6839	0,4519	0,1831

(Com $\mathbf{1}\{m < 0\}$, o ponto $m = 0$, exatamente sobre a fronteira, conta como acerto. É uma convenção: com $\mathbf{1}\{m \leq 0\}$ ele contaria como erro, e nada do que segue muda.)

(b) A *hinge* vale zero exatamente quando $m \geq 1$, isto é $y_i f(\mathbf{x}_i) \geq 1$: o ponto está classificado corretamente **e fora da margem**.

Esses são precisamente os pontos que **não** são vetores de suporte. Como eles contribuem com zero para a soma, movê-los (sem cruzar a margem) não altera o objetivo, e portanto não altera a solução — que é a propriedade do Exercício 1(c), agora vista pelo lado da função de perda em vez da geometria.

A esparsidade da SVM é, portanto, consequência direta de a *hinge* ter uma região **plana em zero**. É o análogo, em perda, do que a quina do losango ℓ_1 faz com os coeficientes no Lasso (Aula 02): um “bico” na função gera soluções exatamente nulas.

(c) **Nunca.** $\log_2(1 + e^{-m}) > 0$ para todo m finito, por menor que seja — em $m = 2$ ainda vale 0,1831, e só tende a zero quando $m \rightarrow \infty$.

A consequência é que, na regressão logística, **todas as observações influenciam o ajuste**, inclusive as que estão longe e do lado certo da fronteira. Não existe “vetor de suporte” na logística: apagar um ponto bem classificado muda a solução, ainda que pouco. Isso a torna mais sensível a observações distantes — em compensação, é o que permite que ela estime probabilidades, já que ela precisa dizer algo sobre todo o espaço, não só perto da fronteira.

(d) Porque a perda 0-1 é **não convexa e descontínua**: ela é constante por partes, com um salto em $m = 0$, e sua derivada é zero em quase todo ponto. Nenhum método baseado em gradiente tem por onde começar, e minimizá-la exatamente é um problema NP-difícil em geral.

Tanto a *hinge* quanto a logística são **cotas superiores convexas** da perda 0-1 — confira na tabela: em cada coluna, as duas últimas linhas são $\geq$ a primeira. (É por isso que a logística aparece aqui em base 2: em logaritmo natural ela vale $\ln 2 \approx 0,69$ em $m = 0$, e fica abaixo de 1 em todo o trecho $-0,54 < m < 0$, onde a 0-1 vale 1. Mudar a base só multiplica a perda por $1/\ln 2$, e não muda o minimizador.) Minimizar uma cota superior convexa é tratável e controla o que interessa. É o mesmo movimento da Aula 02, em que a norma ℓ_1 substitui a ℓ_0 por ser sua relaxação convexa: *trocar o objetivo verdadeiro por uma aproximação convexa que se pode de fato otimizar*.